A Graphical Playground for Haskell

Oliver Jones

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2025

Abstract

Research Ethics Approval

This project obtained approval from the Informatics Research Ethics committee.
Ethics application number: 8978
Date when approval was obtained: 2025-01-08

The participants' information sheet and a consent form are included in the appendix.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Oliver Jones*)

Acknowledgements

I would like to extend my most sincere thanks to my supervisor, Prof. Philip Wadler, for his unwavering guidance and support throughout the project. I would also like to extend my gratitude to all those who tested the software and provided feedback.

Table of Contents

1 Introduction **1**
- 1.1 Aims and Motivations . . . 1
  - 1.1.1 Educational Value . . . 1
  - 1.1.2 Practical Applications . . . 2
- 1.2 Dissertation Outline . . . 2

2 Background **3**
- 2.1 Functional Programming and Haskell . . . 3
- 2.2 Graphics . . . 3
  - 2.2.1 OpenGL . . . 4
  - 2.2.2 Graphics in Haskell . . . 4
  - 2.2.3 Graphics in Web Browsers . . . 4
- 2.3 Existing Work . . . 5
  - 2.3.1 Processing and P5.js . . . 5
  - 2.3.2 Code World . . . 6

3 System Design **7**

4 System Implementation and Challenges **8**

5 Testing and Evaluation **9**

6 Limitations and Future Work **10**

7 Conclusions **11**

Bibliography **12**

A Participants' Information Sheet & Consent Form **14**

B Responses to User Testing & Feedback Survey **18**

Chapter 1

Introduction

This project — A Graphical Playground for Haskell — is a web-based Haskell editor with a built-in graphics library, allowing users to create images and animations using Haskell. The editor provides a simple, interactive interface for writing Haskell code and viewing the output in real time, all within a web browser. The project creates a beginner-friendly experience by providing a visual, interactive environment for learning and experimenting with the language, while also offering a fun and engaging way to create graphics and animations for users of all skill levels.

1.1 Aims and Motivations

There are a variety of motivations for creating this project, including educational benefits for students learning Haskell, as well as practical applications for users interested in creating graphics and animations.

The project aims to provide an experience that is both beginner-friendly and engaging for users of all skill levels. For beginners, the editor offers a visual, interactive environment for learning Haskell, allowing them to see how their code works in real time. For more experienced users, the editor provides a fun and engaging way to create graphics and animations using Haskell, with the ability to experiment and iterate quickly.

1.1.1 Educational Value

Imagine a first-year Computer Science class — a mix of students with varying levels of programming experience, from those who have never written a line of code to those who have been programming for years. Those who do have experience programming have mostly worked with imperative languages like Python or Java, and are now learning Haskell for the first time. The students follow along, trying to understand the code and how it works; but as the program grows more complex, many students start to get lost. Without a way to visualise the program's execution or to see how the different parts of the program interact, they struggle to keep up with the lectures. They strain see how different parts of the program fit together, or how the program's output changes as they modify the code.

This project aims to address this problem by providing students with an interactive, visual environment for learning Haskell. A graphical editor allows students to see how their code works in real time, helping them understand the program's execution and how the different parts of the program interact, while also providing a fun and engaging way to learn Haskell. By providing a web-based service, users never even need to install Haskell on their own computers — they can start coding right away, from any device with a web browser.

1.1.2 Practical Applications

Beyond its educational value, the project also has practical applications for users interested in creating graphics and animations using Haskell. The editor's simple interface and built-in graphics library make it easy to create images and animations, with the ability to experiment and iterate quickly. By including documentation for the graphics library and examples of how to use it, the project aims to make it easy for users to get started creating their own graphics and animations.

1.2 Dissertation Outline

Chapter 2 provides relevant background information on Haskell, functional programming, graphics, and web technologies. This is followed by an exploration of related works, including existing Haskell editors and graphics libraries, placing this project in context.

Chapter 3 describes the design of the project, including the architecture of the editor and the graphics library, as well as the user interface and user experience design. This compares different design choices, explaining the rationale behind the decisions made.

In Chapter 4, the implementation of the project is detailed, illustrating the technologies used and the development process, as well as addressing any challenges faced. This includes the development of both the front-end and back-end components of the project, as well as their integration.

Chapter 5 evaluates the project how successfully it meets its aims and objectives, as well as how well it performs in practice. This includes a discussion of the user testing and feedback survey conducted, as well as any insights gained from the evaluation process.

Chapter 6, meanwhile, discusses the potential areas for future work and improvements to the project. This includes suggestions for how the project could be extended or enhanced in the future, as well as any limitations of the current implementation.

Finally, Chapter 7 concludes the dissertation, summarising the project and reflecting on the process of creating it.

Chapter 2

Background

2.1 Functional Programming and Haskell

Functional programming is a programming paradigm built around the evaluation of mathematical functions (Hudak, 1989). Unlike the more widely used imperative programming, which utilises a series of statements to alter the state of a program and produce a result, functional programming is based on the composition and execution of functions to produce that result. The paradigm is based on lambda calculus, a formal system of computation, developed by Alonzo Church (1936) which is built around function application. Lambda calculus was proved to be equivalent to Turing machines by Alan Turing (1937).

One of the most widely used functional programming languages is Haskell, a statically typed, purely functional programming language, developed in the late 1980s, with the intention of consolidating features from existing functional programming languages into a single, standardised language (Jones, 2003). Haskell's namesake is Haskell Curry, who developed an equivalent system to lambda calculus, combinatory logic, alongside Moses Schönfinkel the 1920s and 1930s (Curry & Feys, 1958).

2.2 Graphics

Computer graphics is a very wide field, encompassing the use of computers for the creation, manipulation, and rendering of images, across both two and three-dimensional domains. Although rudimentary graphics have been around longer, making use of cathode ray tube displays (Braun, 1897), the discipline of computer graphics was first truly established in the 1950s and 60s, with the development of the first viable interactive graphical displays. Since then, the field has grown rapidly, resulting in a plethora of techniques and technologies for creating and rendering graphics.

Two-dimensional graphics are typically stored as raster graphics, where images are represented as a grid of pixels (Noll, 1971). This method of storage is simple, and reflects the same method that modern displays use to render images, but can result in loss of quality when images are scaled. An alternative method of storing graphics, is

to use vector graphics, where shapes are described by mathematical equations, which are then converted into pixels when rendered (Sutherland, 1963). This is a more ideal solution for images which need to be scaled, as the image will not change in quality when resized.

2.2.1 OpenGL

One of the most widely used graphics libraries is OpenGL, a cross-language, cross-platform library developed by Silicon Graphics in the early 1990s (Segal & Akeley, 1997). It supports both 2D and 3D graphics, and is generally used to interact with the graphics processing unit (GPU) to take advantage of hardware acceleration, improving performance particularly for 3D graphics, but can also be used for central processing unit (CPU) rendering. With several hundred procedures available within the API, OpenGL is a both a powerful and versatile library, making it ideal for a wide range of applications, from video games to scientific visualisation.

2.2.2 Graphics in Haskell

2.2.2.1 Gloss

There are a number of libraries and bindings available for Haskell which aid in generating graphics. One of the most popular of these, with a rating on Hackage of 2.75 out of 3, is the Gloss library, which provides a simple interface for creating graphics in Haskell (Lippmeier, 2022). As with many graphics libraries, Gloss uses OpenGL as its rendering engine, but provides a more simple and high-level interface, making it ideal for beginners or for quick sketches. Due to the simplicity of Gloss, there are a very limited number of functions available, which can make it difficult to create more complex graphics.

2.2.2.2 Diagrams

Another popular Haskell graphics library is Diagrams (Yorgey, 2023), which also has a Hackage rating of 2.75, but has fewer downloads than Gloss. This provides a more powerful and flexible interface than Gloss, by allowing the user to choose from one of six backends, including Cairo (Worth & Packard, 2005). The larger API of Diagrams makes it less suitable for beginners, but the flexibility of backends makes the library more versatile for a wider range of applications.

2.2.3 Graphics in Web Browsers

The first web browser, originally named WorldWideWeb before becoming Nexus, was developed by Sir Tim Berners-Lee (1990b), alongside the first web server, and a very basic version of HyperText Markup Language (HTML) (Berners-Lee, 1990a). Web browsers are inherently designed for rendering structured graphical content, and while the first browsers were limited to displaying just HTML, developments in the last three decades, including the introduction of Cascading Style Sheets (CSS) (Lie, 1996),

JavaScript (Eich, 1995), and HTML5 (World Wide Web Consortium, 2008), have allowed for far more complex and interactive graphics.

2.2.3.1 Scalable Vector Graphics (SVGs)

Scalable Vector Graphics (SVGs) use an XML-based format, similar to HTML, to describe two-dimensional vector graphics (World Wide Web Consortium, 2014). Each object in an SVG is appended to the Document Object Model (DOM) of the web page, allowing the browser to automatically re-render the image if any changes are made to the SVG.

2.2.3.2 HTML5 Canvas API

The `canvas` element was first introduced by Apple in 2004 (Hixie, 2004), before being later standardised in the HTML5 specification (Mozilla Developer Connection, 2011) in 2011. By itself, the `canvas` element is nothing but a plain bitmap image. Where it becomes interesting, however, is when it is combined with JavaScript, allowing for the dynamic generation and rendering of graphics. Unlike SVGs, each component that makes up the image is not stored in the DOM, but is rendered directly to the canvas, which can result in better performance once the image is rendered, but requires the whole canvas to be redrawn if any changes are made.

2.2.3.3 WebGL

WebGL brings the capabilities of OpenGL to web browsers, allowing for hardware-accelerated 2D and 3D graphics to be rendered without the need for any plugins (Khronos Group, 2011). Before being standardised by the Khronos Group, WebGL started out as two independent experiments by Mozilla and Opera (Johansson, 2007; Vukićević, 2007). WebGL renders inside the `canvas` element, and is based on OpenGL ES 2.0, a subset of OpenGL which is designed for embedded systems (Munshi & Leech, 2010).

2.3 Existing Work

2.3.1 Processing and P5.js

Processing is a programming language and integrated development environment (IDE) based on Java (with bindings available for JavaScript, Python and Ruby), allowing for the creation of graphics and animations (Processing Foundation, 2018). P5.js is the official JavaScript binding for Processing, which uses the web browser to display graphics via the HTML `canvas` element (McCarthy, 2013a). Processing and P5.js sketches are built around two core functions: `setup()` which is called once when the program starts; and `draw()` which is called repeatedly to update the canvas. While Processing uses OpenGL as its rendering engine, P5.js uses the HTML5 Canvas API, with WebGL support available as an opt-in feature (McCarthy, 2013b) to improve performance and enable 3D graphics.

While there is no official support for Haskell from the Processing Foundation, there have been some unofficial bindings published on Hackage. One such package is processing by Díaz (2016), which uses p5.js as a backend, producing its output using the HTML5 `canvas` element, but this was last updated in 2016, and has seemingly been abandoned. Another package, processing-for-haskell by Kholomiov (2016), was last updated in 2022, and implements almost the entire Processing API in Haskell, using OpenGL as the backend, just as the original Processing does, but does not support generating graphics in web browsers.

2.3.2 Code World

Code World is an educational website distributed by Google and created by Chris Smith (2014), which provides an environment for creating simple graphics using Haskell. There are three versions of Code World available: one which uses an educational version of Haskell 98; one which uses a full version of Haskell 2010; and one which uses a block editor, designed for beginners. While Code World is a useful tool for teaching Haskell, basic programming concepts, and simple graphics concepts, it lacks certain features which could allow for more creativity from the programmer. For example, the ability to see the results of changes in real-time can be highly beneficial, as it allows the programmer to discover effects which they may not have considered before (Victor, 2012). This can be achieved through the implementation of livelits, a feature which allows programmers to alter the values of variables, and see the results in real-time, without having to recompile the program (Omar et al., 2021).

Chapter 3

System Design

Chapter 4

System Implementation and Challenges

Chapter 5

Testing and Evaluation

Chapter 6

Limitations and Future Work

Chapter 7

Conclusions

Bibliography

Berners-Lee, S. T. (1990a). *Information management: A proposal*. https://cds.cern.ch/record/369245/files/dd-89-001.pdf

Berners-Lee, S. T. (1990b). *The worldwideweb browser*. https://www.w3.org/People/Berners-Lee/WorldWideWeb.html

Braun, F. (1897). Ueber ein verfahren zur demonstration und zum studium des zeitlichen verlaufes variabler ströme [on a process for the display and study of the course in time of variable currents]. *Annalen der Physik und Chemie*, *60*.

Church, A. (1936). A formulation of the simple theory of types. *Journal of Symbolic Logic*, *5*. https://www.cambridge.org/core/journals/journal-of-symbolic-logic/article/abs/formulation-of-the-simple-theory-of-types/85B3666C7DD81A4F66966A399364B44B

Curry, H. B., & Feys, R. (1958). Combinatory logic. *North-Holland Publishing Company*.

Díaz, D. (2016). *Processing [haskell]*. https://hackage.haskell.org/package/processing

Eich, B. (1995). *Javascript*. TODO

Hixie, I. (2004). *Extending html*. http://ln.hixie.ch/?start=1089635050&count=1

Hudak, P. (1989). Conception, evolution, and application of functional programming languages. *ACM Computing Surveys*, *21*. https://dl.acm.org/doi/pdf/10.1145/72551.72554

Johansson, T. (2007). *Taking the canvas to another dimension*. https://web.archive.org/web/20071117170113/http://my.opera.com/timjoh/blog/2007/11/13/taking-the-canvas-to-another-dimension

Jones, S. P. (2003). *Haskell 98 language and libraries: The revised report* (Technical Report). Cambridge University Press. https://www.haskell.org/onlinereport/preface-jfp.html

Kholomiov, A. (2016). *Processing for haskell*. https://hackage.haskell.org/package/processing-for-haskell

Khronos Group. (2011). *Final webgl 1.0 specification*. https://www.khronos.org/news/press/khronos-releases-final-webgl-1.0-specification

Lie, H. W. (1996). *Cascading html style sheets — a proposal*. https://www.w3.org/People/howcome/p/cascade.html

Lippmeier, B. (2022). *Gloss: Painless 2d vector graphics, animations and simulations*. https://hackage.haskell.org/package/gloss

McCarthy, L. (2013a). *P5.js*. https://p5js.org/about/

McCarthy, L. (2013b). *P5.js reference — webgl*. https://p5js.org/reference/p5/WEBGL/

Mozilla Developer Connection. (2011). *Htmlcanvaselement*. https://web.archive.org/web/20110604062413/https://developer.mozilla.org/en/DOM/HTMLCanvasElement

Munshi, A., & Leech, J. (2010). *Opengl es common profile specification version 2.0.25 (full specification)*. https://registry.khronos.org/OpenGL/specs/es/2.0/es_full_spec_2.0.pdf

Noll, A. M. (1971). Scanned-display computer graphics. *Communications of the ACM*, *14*. https://dl.acm.org/doi/10.1145/362566.362567

Omar, C., Moon, D., Blinn, A., Voysey, I., Collins, N., & Chugh, R. (2021). Filling typed holes with live guis. *International Conference on Programming Language Design and Implementation*. https://hazel.org/papers/livelits-paper.pdf

Processing Foundation. (2018). A modern prometheus. *Medium*. https://medium.com/processing-foundation/a-modern-prometheus-59aed94abe85

Segal, M., & Akeley, K. (1997). *The opengl graphics system: A specification (version 1.1)*. https://registry.khronos.org/OpenGL/specs/gl/glspec11.pdf

Smith, C. (2014). *Codeworld*. https://github.com/google/codeworld

Sutherland, I. (1963). *Sketchpad: A man-machine graphical communication system* [Ph.D. Thesis]. Massachusetts Institute of Technology. https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-574.pdf

Turing, A. (1937). Computability and λ-definability. *Journal of Symbolic Logic*, *2*. https://www.jstor.org/stable/2268280

Victor, B. (2012). *Inventing on principle*. https://www.youtube.com/watch?v=PUv66718DII

Vukićević, V. (2007). *Canvas 3d: Gl power, web-style*. https://web.archive.org/web/20110717224855/http://blog.vlad1.com/2007/11/26/canvas-3d-gl-power-web-style/

World Wide Web Consortium. (2008). *Html5*. TODO

World Wide Web Consortium. (2014). *Secret origin of svg*. https://www.w3.org/Graphics/SVG/WG/wiki/Secret_Origin_of_SVG

Worth, C., & Packard, K. (2005). *Cairo*. https://www.cairographics.org

Yorgey, B. (2023). *Diagrams: Embedded domain-specific language for declarative vector graphics*. https://hackage.haskell.org/package/diagrams

Appendix A

Participants' Information Sheet & Consent Form

The following pages contain the participants' information sheet and consent form, verbatim as presented to participants.

Participant Information Sheet

Project title:	A Graphical Playground for Haskell
Principal investigator:	Professor Philip Wadler
Researcher collecting data:	Oliver Jones

This study was certified according to the Informatics Research Ethics Process, reference number 102908. Please take time to read the following information carefully. You should keep this page for your records.

Who are the researchers?

Oliver Jones and Professor Philip Wadler

What is the purpose of the study?

To assess the usability of the system, allowing me to evaluate how successful the project is.

Why have I been asked to take part?

The research target group is people with varying backgrounds in computer science and functional programming.

Do I have to take part?

No – participation in this study is entirely up to you. You can withdraw from the study at any time, without giving a reason. Your rights will not be affected. If you wish to withdraw, contact the PI. We will stop using your data in any publications or presentations submitted after you have withdrawn consent. However, we will keep copies of your original consent, and of your withdrawal request.

What will happen if I decide to take part?

You will be asked to complete a series of tasks using the system. For each task, you will be asked to answer a few questions, rating various aspects of the system's usability. This will be done remotely, with questions being answered via an online survey.

Are there any risks associated with taking part?

There are no significant risks associated with participation.

Are there any benefits associated with taking part?

No

What will happen to the results of this study?

The results of this study may be summarised in published articles, reports and presentations. Quotes or key findings will be anonymized: We will remove any information that could, in our assessment, allow anyone to identify you. With your consent, information can also be used for future research. Your data may be archived for a minimum of 2 years.

Data protection and confidentiality.

Your data will be processed in accordance with Data Protection Law. All information collected about you will be kept strictly confidential. Your data will be referred to by a unique participant number rather than by name. Your data will only be viewed by Oliver Jones.

All electronic data will be stored on a password-protected encrypted computer, on the School of Informatics' secure file servers, or on the University's secure encrypted cloud storage services (DataShare, ownCloud, or Sharepoint) and all paper records will be stored in a locked filing cabinet in the PI's office. Your consent information will be kept separately from your responses in order to minimise risk.

What are my data protection rights?

The University of Edinburgh is a Data Controller for the information you provide. You have the right to access information held about you. Your right of access can be exercised in accordance Data Protection Law. You also have other rights including rights of correction, erasure and objection. For more details, including the right to lodge a complaint with the Information Commissioner's Office, please visit www.ico.org.uk. Questions, comments and requests about your personal data can also be sent to the University Data Protection Officer at dpo@ed.ac.uk.
For general information about how we use your data, go to: edin.ac/privacy-research

Who can I contact?

If you have any further questions about the study, please contact the lead researcher, Oliver Jones (s2153980@ed.ac.uk).

If you wish to make a complaint about the study, please contact inf-ethics@inf.ed.ac.uk. When you contact us, please provide the study title and detail the nature of your complaint.

Updated information.

If the research project changes in any way, an updated Participant Information Sheet will be made available on http://web.inf.ed.ac.uk/infweb/research/study-updates.

Consent

By proceeding with the study, I agree to all of the following statements:

- I have read and understood the above information.
- I understand that my participation is voluntary, and I can withdraw at any time.
- I consent to my anonymised data being used in academic publications and presentations.
- I allow my data to be used in future ethically approved research.

  Take me to the survey

Appendix B

Responses to User Testing & Feedback Survey

The following pages contain the responses to the user testing and feedback survey, as collected from participants.